

**Модульна контрольна робота  
з навчальної дисципліни**

**«Об'єктно-орієнтоване програмування»**

**Виконав:**

**студент групи alk-43**

**Устич Максим**

**Київ 2026р.**

### *1. Запишіть загальні форми операторів try, catch і throw. Опишіть їх функції.*

У мові програмування C++ для оброблення виняткових ситуацій використовуються оператори `try`, `catch` і `throw`.

Оператор `try` визначає блок коду, у якому може виникнути помилка під час виконання програми.

Загальна форма оператора `try`:

```
try
{
    // програмний код
}
```

Оператор `throw` використовується для створення або передачі виняткової ситуації.

Загальна форма оператора `throw`:

```
throw значення;
```

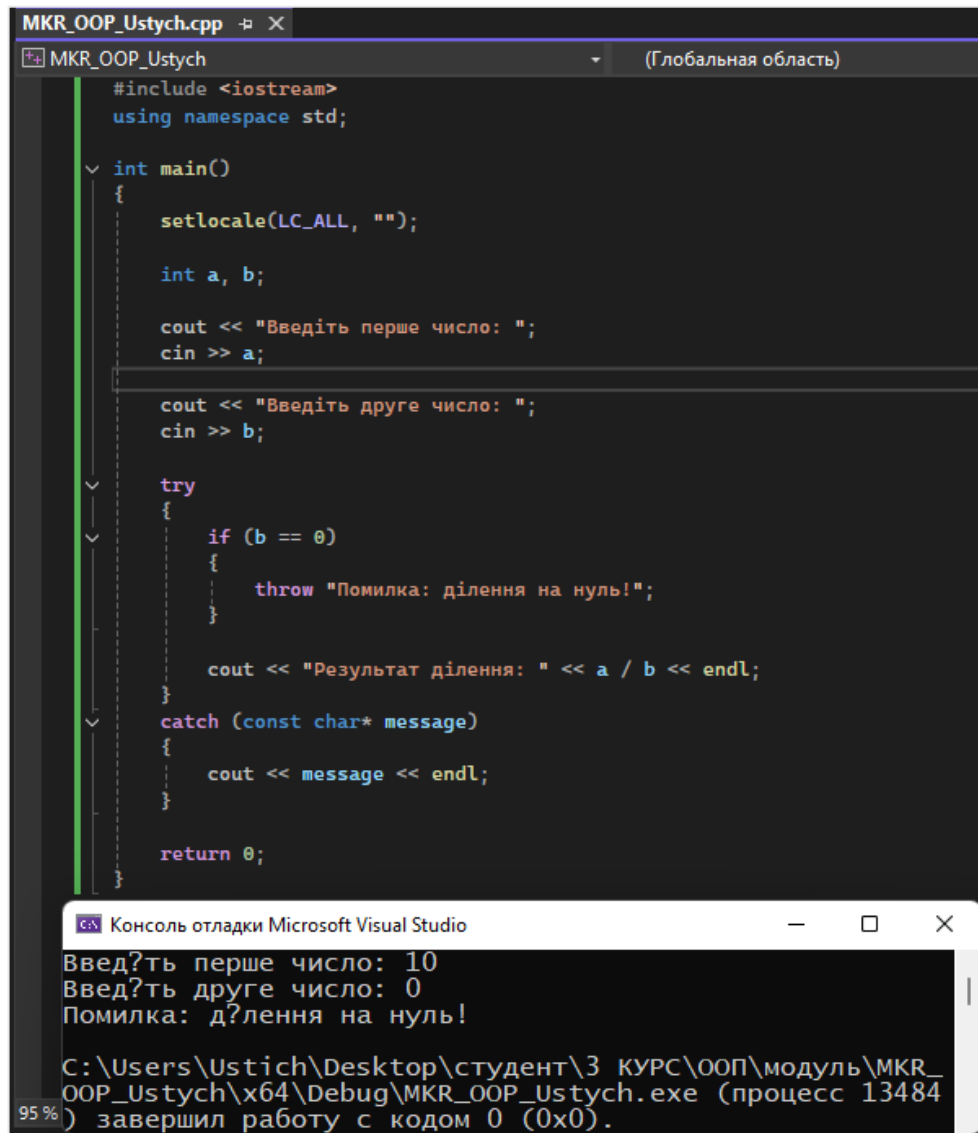
Оператор `catch` призначений для перехоплення та оброблення винятків.

Загальна форма оператора `catch`:

```
catch(тип параметр)
{
    // оброблення винятку
}
```

Сумісне використання операторів `try`, `catch` і `throw` дозволяє програмі коректно обробляти помилки та уникати аварійного завершення програми.

## Приклад програмної реалізації:



```
MKR_OOP_Ustych.cpp  X
MKR_OOP_Ustych  (Глобальная область)

#include <iostream>
using namespace std;

int main()
{
    setlocale(LC_ALL, "");

    int a, b;

    cout << "Введіть перше число: ";
    cin >> a;

    cout << "Введіть друге число: ";
    cin >> b;

    try
    {
        if (b == 0)
        {
            throw "Помилка: ділення на нуль!";
        }

        cout << "Результат ділення: " << a / b << endl;
    }
    catch (const char* message)
    {
        cout << message << endl;
    }

    return 0;
}
```

Консоль отладки Microsoft Visual Studio

```
Введ?ть перше число: 10
Введ?ть друге число: 0
Помилка: д?лення на нуль!

C:\Users\Ustich\Desktop\студент\3 КУРС\ООП\модуль\MKR_
ООП_Ustych\x64\Debug\MKR_OOP_Ustych.exe (процесс 13484
) завершил работу с кодом 0 (0x0).
```

Рис. 1 – Програмна реалізація операторів `try`, `catch` і `throw` у середовищі *Visual Studio*.

У наведеній програмі здійснюється перевірка ділення на нуль. Якщо користувач вводить нуль як друге число, оператор `throw` генерує виняткову ситуацію. Оператор `catch` перехоплює виняток і виводить повідомлення про помилку «Помилка: ділення на нуль!». Таким чином забезпечується коректне оброблення помилки без аварійного завершення програми.

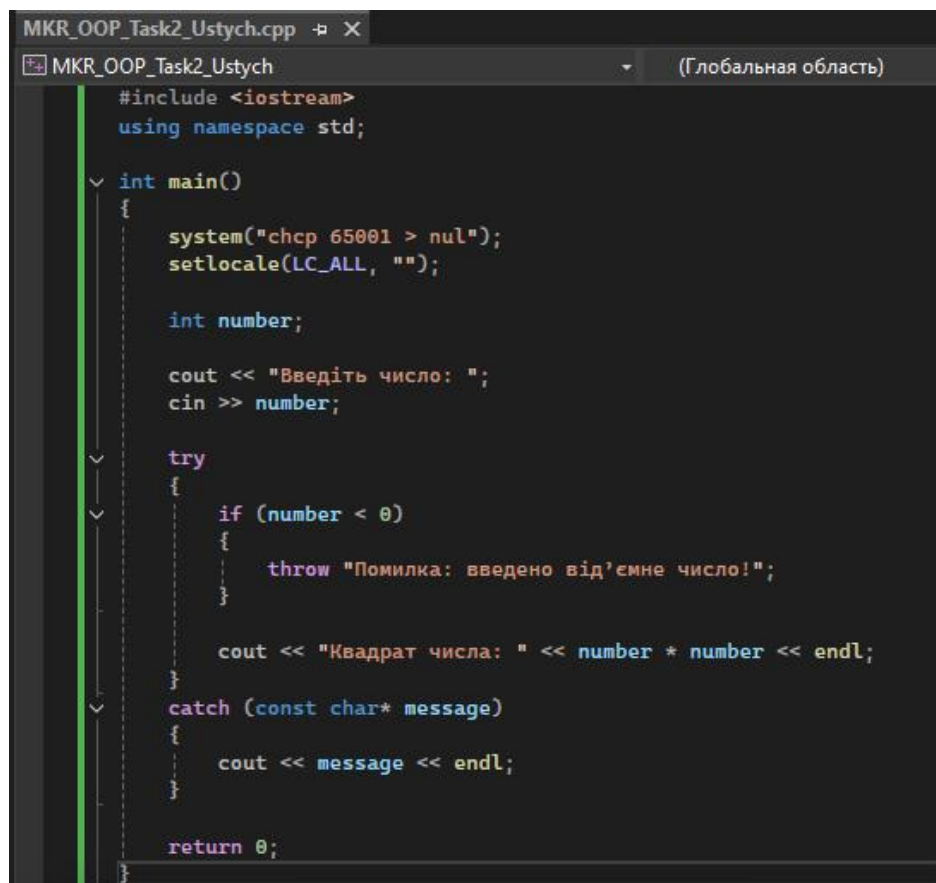
## 2. Поясніть на конкретному прикладі, яким чином сумісна робота операторів `try`, `catch` і `throw` забезпечує в C++ оброблення виняткових ситуацій?

У мові програмування C++ оператори `try`, `catch` і `throw` працюють спільно для виявлення, передачі та оброблення помилок під час виконання програми.

Оператор `try` визначає блок коду, у якому може виникнути помилка. Якщо під час виконання цього блоку виникає виняткова ситуація, використовується оператор `throw`, який генерує виняток. Після цього керування передається оператору `catch`, який перехоплює виняток і виконує його оброблення.

Завдяки цьому програма не завершується аварійно, а продовжує роботу після оброблення помилки.

### Приклад програмної реалізації:



```
MKR_OOP_Task2_Ustych.cpp  X
MKR_OOP_Task2_Ustych (Глобальная область)

#include <iostream>
using namespace std;

int main()
{
    system("chcp 65001 > nul");
    setlocale(LC_ALL, "");

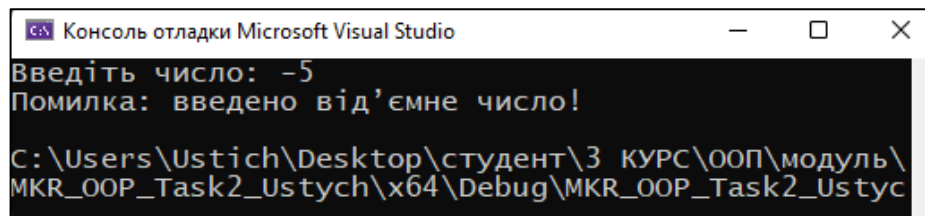
    int number;

    cout << "Введіть число: ";
    cin >> number;

    try
    {
        if (number < 0)
        {
            throw "Помилка: введено від'ємне число!";
        }

        cout << "Квадрат числа: " << number * number << endl;
    }
    catch (const char* message)
    {
        cout << message << endl;
    }

    return 0;
}
```



```
Консоль отладки Microsoft Visual Studio
Введіть число: -5
Помилка: введено від'ємне число!

C:\Users\Ustich\Desktop\студент\3 КУРС\ООП\модуль\
MKR_OOP_Task2_Ustych\x64\Debug\MKR_OOP_Task2_Ustyc
```

Рис. 2 – Приклад сумісної роботи операторів `try`, `catch` і `throw` у C++.

У наведеній програмі користувач вводить число. Якщо введене число є від'ємним, оператор `throw` генерує виняткову ситуацію з повідомленням про помилку. Оператор `catch` перехоплює цей виняток і виводить повідомлення «Помилка: введено від'ємне число!». Якщо помилка відсутня, програма обчислює квадрат числа.

Таким чином забезпечується коректне оброблення виняткових ситуацій у програмі.

### ***3. Чи можна використовувати `throw`, якщо виконання програми не проходить через блок `try`? Наведіть конкретний приклад.***

У мові програмування C++ оператор `throw` можна використовувати і поза блоком `try`. Проте якщо виняток не буде перехоплений оператором `catch`, програма аварійно завершить свою роботу.

Якщо виконання програми не проходить через блок `try`, виняткова ситуація не буде оброблена. У такому випадку система повідомить про помилку та завершить виконання програми.

#### **Приклад програмної реалізації:**

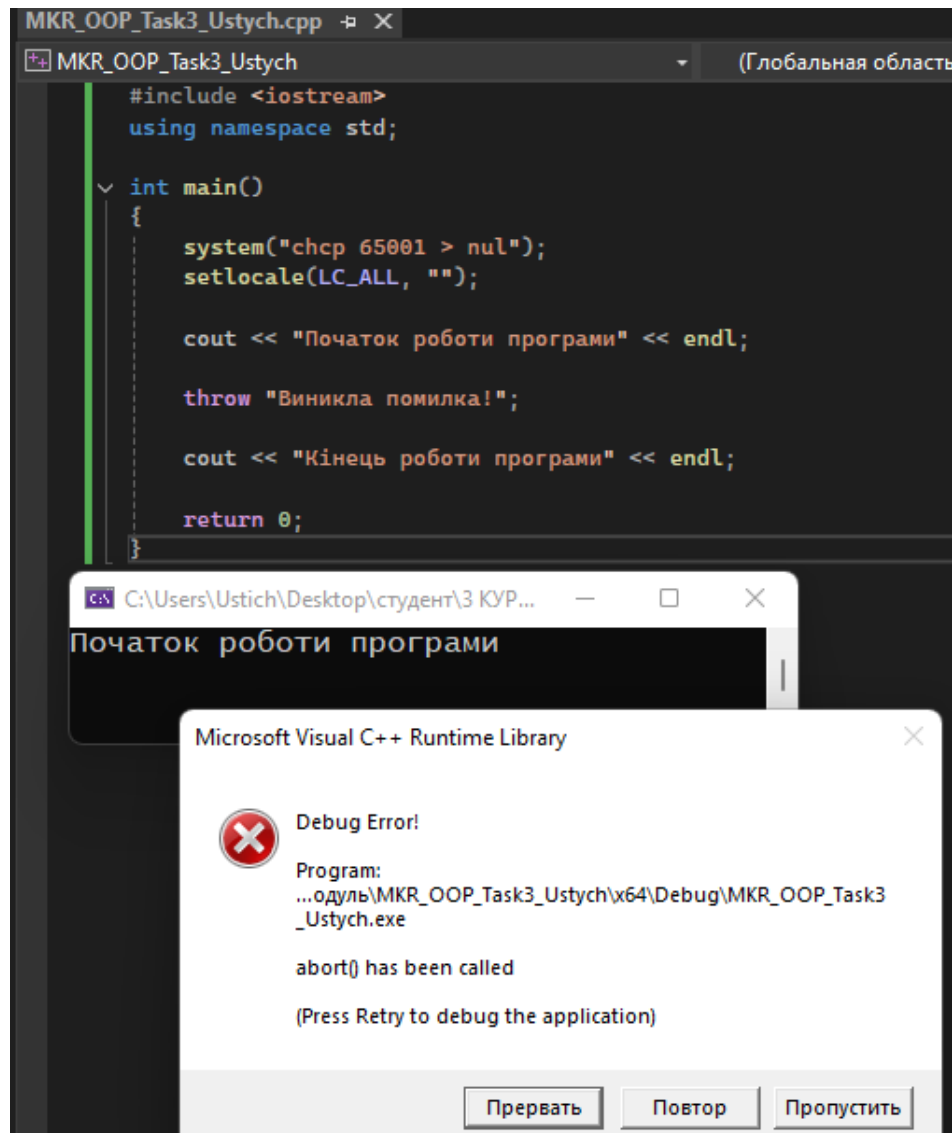


Рис. 3 – Використання оператора throw без блоку try у C++.

**4. Наведіть декілька переваг використання функцій введення-виведення C++ порівняно із системою введення-виведення мови C. Наведіть конкретні приклади.**

У мові програмування C++ для введення та виведення даних використовуються потокові об'єкти cin і cout, які є більш зручними та безпечними порівняно з функціями scanf() і printf() мови C.

Основні переваги системи введення-виведення C++:

### 1. Простота використання.

У C++ не потрібно вказувати специфікатори формату (%d, %f, %s), як у мові C.

### 2. Підтримка різних типів даних.

Оператори << та >> автоматично визначають тип даних.

### 3. Вища безпеність.

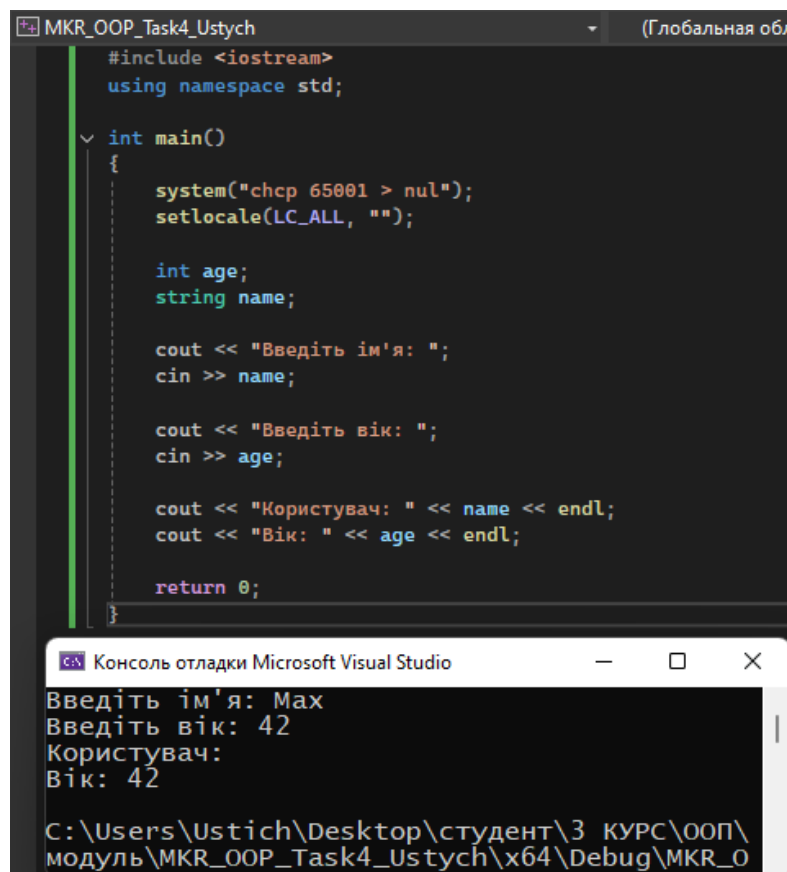
Потокове введення-виведення зменшує ймовірність помилок під час роботи з пам'яттю та форматами даних.

### 4. Зручне перевантаження операторів.

У C++ можна перевантажувати оператори введення та виведення для власних класів.

### 5. Краща інтеграція з об'єктно-орієнтованим програмуванням.

## Приклад використання введення-виведення у C++:



The image shows a screenshot of a C++ program in Visual Studio. The code is in a file named MKR\_OOP\_Task4\_Ustych. It includes the <iostream> header and uses the std namespace. The main function contains the following code:

```
#include <iostream>
using namespace std;

int main()
{
    system("chcp 65001 > nul");
    setlocale(LC_ALL, "");

    int age;
    string name;

    cout << "Введіть ім'я: ";
    cin >> name;

    cout << "Введіть вік: ";
    cin >> age;

    cout << "Користувач: " << name << endl;
    cout << "Вік: " << age << endl;

    return 0;
}
```

The output window shows the program's execution:

```
Введіть ім'я: Max
Введіть вік: 42
Користувач:
Вік: 42

C:\Users\Ustich\Desktop\студент\3 КУРС\ООП\
модуль\MKR_OOP_Task4_Ustych\x64\Debug\MKR_O
```

*Рис. 4 – Приклад використання потоків введення-виведення у C++.*

Приклад використання введення-виведення у мові C:

```
#include <stdio.h>

int main()
{
    int age;
    char name[20];

    printf("Введіть ім'я: ");
    scanf("%s", name);

    printf("Введіть вік: ");
    scanf("%d", &age);

    printf("Користувач: %s\n", name);
    printf("Вік: %d\n", age);

    return 0;
}
```

У наведеному прикладі використання `cin` і `cout` є більш простим та зрозумілим порівняно з `scanf()` і `printf()` мови C. Потокова система введення-виведення C++ забезпечує кращу читабельність коду, автоматичну підтримку типів даних та зручну інтеграцію з об'єктно-орієнтованим програмуванням.

***5. Які функції, що визначають параметри формату ви знаєте? Яке їх призначення?***

***Наведіть конкретні приклади застосування.***

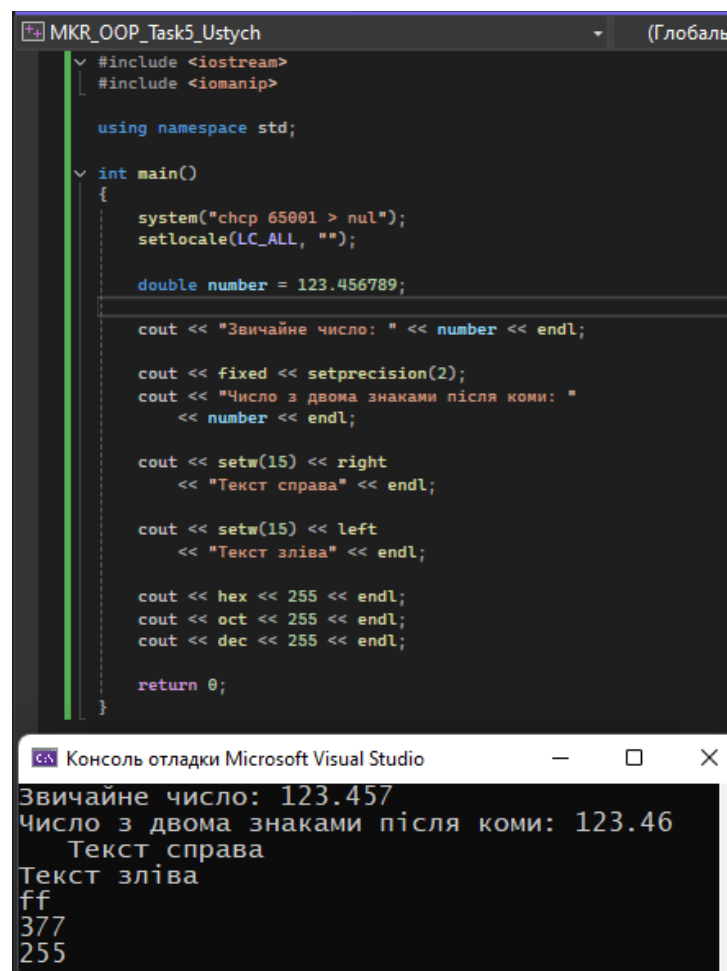


У мові програмування C++ для форматування введення та виведення даних використовуються спеціальні функції та маніпулятори потоків. Вони дозволяють змінювати вигляд чисел, тексту та інших даних під час виведення на екран.

Основні функції та маніпулятори форматування:

1. `setw()` – встановлює ширину поля виведення.
2. `setprecision()` – встановлює кількість цифр після коми.
3. `fixed` – виводить число у звичайному десятковому форматі.
4. `left` та `right` – вирівнювання тексту по лівому або правому краю.
5. `showpoint` – примусово показує десяткову крапку.
6. `hex`, `oct`, `dec` – виведення числа у шістнадцятковій, вісімковій або десятковій системі числення.
7. `endl` – перехід на новий рядок.

### Приклад програмної реалізації:



```
MKR_OOP_Task5_Ustych (Глобальн...
#include <iostream>
#include <iomanip>

using namespace std;

int main()
{
    system("chcp 65001 > nul");
    setlocale(LC_ALL, "");

    double number = 123.456789;

    cout << "Звичайне число: " << number << endl;

    cout << fixed << setprecision(2);
    cout << "Число з двома знаками після коми: "
        << number << endl;

    cout << setw(15) << right
        << "Текст справа" << endl;

    cout << setw(15) << left
        << "Текст зліва" << endl;

    cout << hex << 255 << endl;
    cout << oct << 255 << endl;
    cout << dec << 255 << endl;

    return 0;
}
```

Консоль отладки Microsoft Visual Studio

```
Звичайне число: 123.457
Число з двома знаками після коми: 123.46
Текст справа
Текст зліва
ff
377
255
```

*Рис. 5 – Приклад використання функцій форматування у C++.*

У наведеній програмі використано маніпулятори форматування `setw()`, `setprecision()`, `fixed`, `left`, `right`, `hex`, `oct` і `dec`. Вони дозволяють змінювати вигляд даних під час виведення та роблять результати роботи програми більш зрозумілими і зручними для користувача.

***6. У чому полягає зручність для клієнта замість безпосереднього створення потрібних об'єктів створювати для цього фабрику? Поясніть відповідь на конкретному прикладі.***

Патерн `Factory Method` використовується для створення об'єктів без безпосереднього використання оператора `new` у клієнтському коді.

Основна перевага фабрики полягає в тому, що клієнту не потрібно знати деталі створення об'єкта. Фабрика самостійно створює потрібний об'єкт і повертає його клієнту.

Переваги використання фабрики:

1. Спрощення створення об'єктів.
2. Приховування деталей реалізації.
3. Зменшення залежності клієнтського коду від конкретних класів.
4. Зручність розширення програми новими типами об'єктів.
5. Покращення підтримки та масштабування програмного забезпечення.

**Приклад програмної реалізації:**

```
MKR_OOP_Task6_Ustych AnimalFact
#include <iostream>

using namespace std;

class Animal
{
public:
    virtual void sound() = 0;
};
```

```
class Cat : public Animal
{
public:
    void sound() override
    {
        cout << "Кит нявкає" << endl;
    }
};

class AnimalFactory
{
public:
    static Animal* createAnimal(int type)
    {
        if (type == 1)
            return new Dog();

        if (type == 2)
            return new Cat();

        return nullptr;
    }
};

int main()
{
    system("chcp 65001 > nul");
    setlocale(LC_ALL, "");

    Animal* animal;

    animal = AnimalFactory::createAnimal(1);

    animal->sound();

    delete animal;

    return 0;
}
```

```
Консоль отладки Microsoft Visual Studio
Собака гавкає
C:\Users\Ustich\Desktop\студент\3 КУРС\ООП\модуль\MKR_OOP_Task6_Ustych\x64\Debug\MKR_OOP_Task6_Ustych.exe (процесс 11080) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно...
```

У наведеному прикладі клас `AnimalFactory` створює об'єкти класів `Dog` і `Cat`. Клієнтський код не використовує оператор `new` безпосередньо, а звертається до фабрики для створення потрібного об'єкта. Це спрощує роботу з об'єктами та робить програму більш гнучкою для подальшого розширення.

***7. Яким чином впроваджуються і трансформуються залежності при використанні патерну Dependency Injection? Поясніть відповідь на конкретних прикладах.***

Патерн `Dependency Injection` використовується для передачі об'єктів-залежностей іншому класу ззовні, а не створення їх усередині класу.

Основна ідея цього патерну полягає у тому, що клас не створює необхідні йому об'єкти самостійно за допомогою оператора `new`, а отримує їх через конструктор, метод або властивість.

Завдяки цьому зменшується залежність між класами та підвищується гнучкість програмного забезпечення.

Основні переваги `Dependency Injection`:

1. Зменшення зв'язності між класами.
2. Спрощення тестування програм.
3. Можливість легко замінювати компоненти програми.
4. Покращення підтримки та масштабування програмного забезпечення.

**Приклад програмної реалізації:**

```
MKR_OOP_Task7_Ustych (Глобальн...  
  
#include <iostream>  
  
using namespace std;  
  
class Engine  
{  
public:  
    void start()  
    {  
        cout << "Двигун запущено" << endl;  
    }  
};  
  
class Car  
{  
private:  
    Engine* engine;  
public:  
    Car(Engine* e)  
    {  
        engine = e;  
    }  
  
    void move()  
    {  
        engine->start();  
        cout << "Автомобіль рухається" << endl;  
    }  
};  
  
int main()  
{  
    system("chcp 65001 > nul");  
    setlocale(LC_ALL, "");  
  
    Engine engine;  
  
    Car car(&engine);  
  
    car.move();  
  
    return 0;  
}
```

Консоль отладки Microsoft Visual Studio

Двигун запущено  
Автомобіль рухається

C:\Users\Ustych\Desktop\студент\3 КУРС\ООП\модуль\MKR\_OOP\_Task7\_Ustych\x64\Debug\MKR\_OOP\_Task7\_Ustych.exe (процесс 13620) завершил работу с кодом 0 (0x0)

Рис. 7 – Приклад використання патерну *Dependency Injection* у C++.

У наведеному прикладі клас `Car` залежить від класу `Engine`. Проте об'єкт двигуна не створюється всередині класу `Car`, а передається через

конструктор. Таким чином здійснюється впровадження залежності. Це дозволяє легко замінити один тип двигуна на інший без зміни коду класу `Car`, що робить програму більш гнучкою та зручною для підтримки.

#### ***8. Поясніть на конкретних прикладах, які наслідки матиме поява нового похідного конкретного класу елементів відповідно патерну Visitor?***

Патерн Visitor використовується для додавання нових операцій до групи об'єктів без зміни їхніх класів.

Основна ідея патерну полягає у відокремленні алгоритмів від структури об'єктів. Для цього створюється окремий клас `Visitor`, який виконує певні операції над елементами

Поява нового похідного конкретного класу елементів у патерні Visitor призводить до необхідності змінювати всі класи `Visitor`. Це є основним недоліком даного патерну.

Наприклад, якщо до системи додається новий клас `Triangle`, тоді в усіх класах `Visitor` потрібно реалізувати новий метод `visitTriangle()`.

Таким чином:

- нові операції додавати легко;
- нові типи елементів додавати складніше.

**Приклад програмної реалізації:**

```
MKR_OOP_Task8_Ustych (Глобальн
#include <iostream>

using namespace std;

class Circle;
class Square;

class Visitor
{
public:
    virtual void visitCircle(Circle* c) = 0;
    virtual void visitSquare(Square* s) = 0;
};

class Shape
{
public:
    virtual void accept(Visitor* v) = 0;
};

class Circle : public Shape
{
public:
    void accept(Visitor* v) override
    {
```

```
class Square : public Shape
{
public:
    void accept(Visitor* v) override
    {
        v->visitSquare(this);
    }
};

class DrawVisitor : public Visitor
{
public:
    void visitCircle(Circle* c) override
    {
        cout << "Малювання круга" << endl;
    }

    void visitSquare(Square* s) override
    {
        cout << "Малювання квадрата" << endl;
    }
};

int main()
{
    system("chcp 65001 > nul");
    setlocale(LC_ALL, "");

    DrawVisitor visitor;

    Circle circle;
    Square square;

    circle.accept(&visitor);
    square.accept(&visitor);

    return 0;
}
```

Консоль отладки Microsoft Visual Studio

Малювання круга  
Малювання квадрата

C:\Users\Ustich\Desktop\студент\3 КУРС\ООП\модуль\MKR\_OOP\_Task8\_Ustych\...  
64\Debug\MKR\_OOP\_Task8\_Ustych.exe (процесс 9724) завершил работу с кодом  
0 (0x0)

Рис. 8 – Приклад використання патерну Visitor у C++.

У наведеному прикладі клас `Visitor` містить методи для роботи з різними типами фігур. Якщо у програмі з'явиться новий похідний клас, наприклад `Triangle`, тоді доведеться додавати новий метод до базового класу `Visitor` і реалізовувати його у всіх похідних класах `Visitor`. Це ускладнює підтримку програми при додаванні нових типів елементів.



### *1. Чому інтерфейс з великою кількістю різноманітних методів краще розділити на декілька менших інтерфейсів? Наведіть конкретний приклад.*

У об'єктно-орієнтованому програмуванні великий інтерфейс із великою кількістю різноманітних методів рекомендується розділяти на декілька менших спеціалізованих інтерфейсів.

Такий підхід відповідає принципу ISP – принципу розділення інтерфейсів.

Суть цього принципу полягає у тому, що класи не повинні залежати від методів, які вони не використовують.

Переваги розділення інтерфейсів:

1. Зменшення залежностей між класами.
2. Спрощення підтримки та модифікації програмного забезпечення.
3. Покращення читабельності коду.
4. Зручність тестування програм.
5. Підвищення гнучкості та масштабованості системи.

Наприклад, якщо створити один великий інтерфейс для роботи з принтером, сканером і факсом, то деякі класи будуть змушені реалізовувати непотрібні їм методи.

### **Приклад неправильного підходу:**

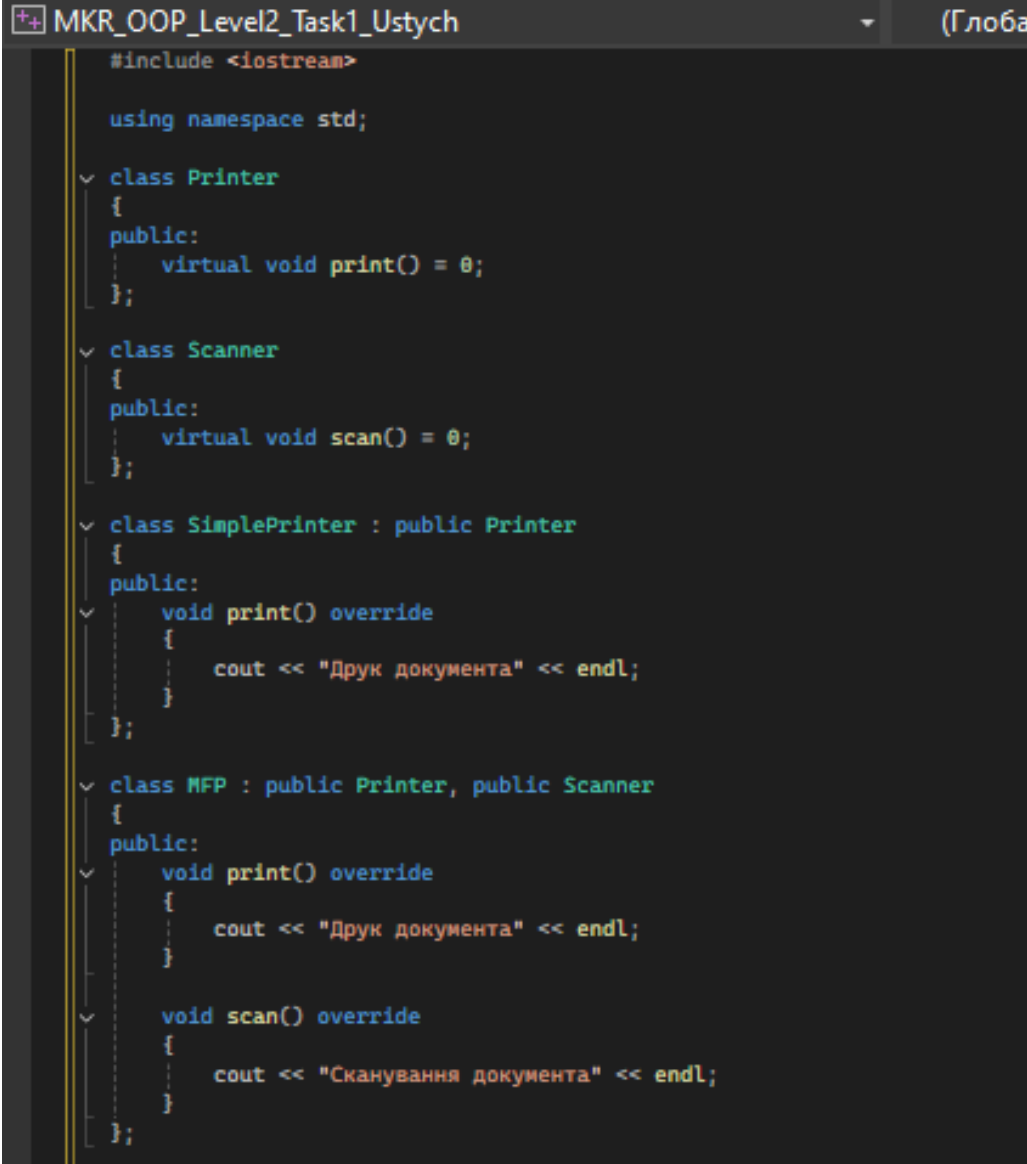
```
class Device
{
public:
```

```
virtual void print() = 0;
virtual void scan() = 0;
virtual void fax() = 0;
};
```

У такому випадку звичайний принтер повинен реалізовувати методи `scan()` і `fax()`, хоча вони йому не потрібні.

Правильним рішенням є розділення інтерфейсів.

**Приклад програмної реалізації:**



```
++ MKR_OOP_Level2_Task1_Ustych (Глоба
#include <iostream>
using namespace std;

class Printer
{
public:
    virtual void print() = 0;
};

class Scanner
{
public:
    virtual void scan() = 0;
};

class SimplePrinter : public Printer
{
public:
    void print() override
    {
        cout << "Друк документа" << endl;
    }
};

class MFP : public Printer, public Scanner
{
public:
    void print() override
    {
        cout << "Друк документа" << endl;
    }

    void scan() override
    {
        cout << "Сканування документа" << endl;
    }
};
```

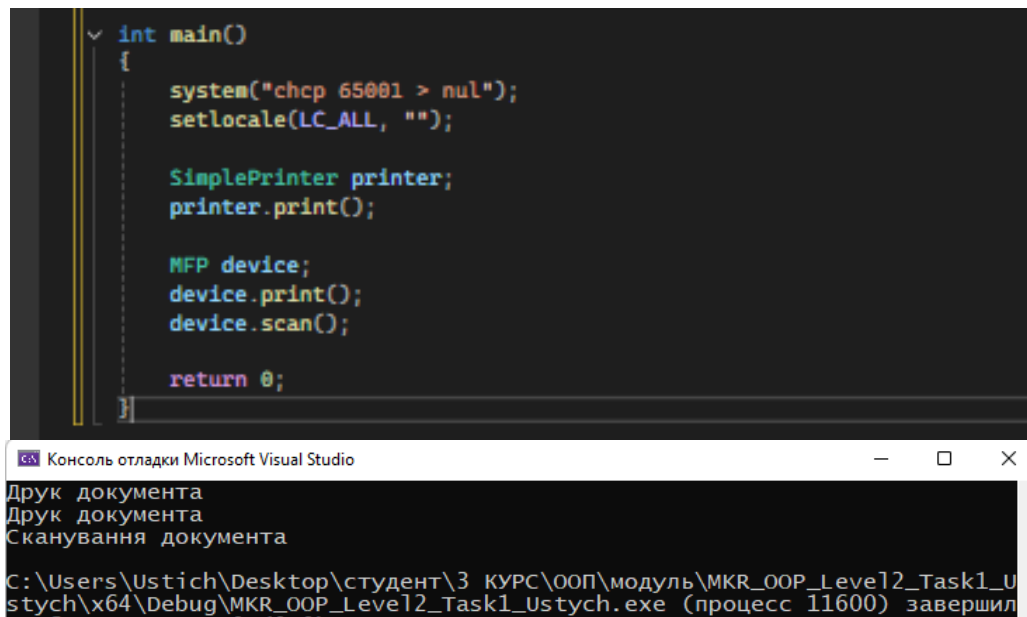


Рис. 9 – Приклад розділення великого інтерфейсу на декілька менших у C++.

У наведеному прикладі великий інтерфейс було розділено на окремі інтерфейси `Printer` і `Scanner`. Завдяки цьому клас `SimplePrinter` реалізує лише необхідний метод `print()`, а клас `MFP` реалізує обидва інтерфейси. Такий підхід спрощує структуру програми та відповідає принципу розділення інтерфейсів.

## ***2. Яку інформацію і кому можна передавати через параметр(u) методу Update в патерні Спостерігач? Наведіть конкретний приклад програмної реалізації.***

Патерн Observer (Спостерігач) використовується для організації зв'язку між об'єктами за принципом «один до багатьох». Коли стан одного об'єкта змінюється, усі залежні об'єкти автоматично отримують повідомлення про ці зміни.

Метод `Update()` використовується для передачі інформації від об'єкта `Subject` (суб'єкта) до об'єктів `Observer` (спостерігачів).

Через параметри методу `Update()` можна передавати:

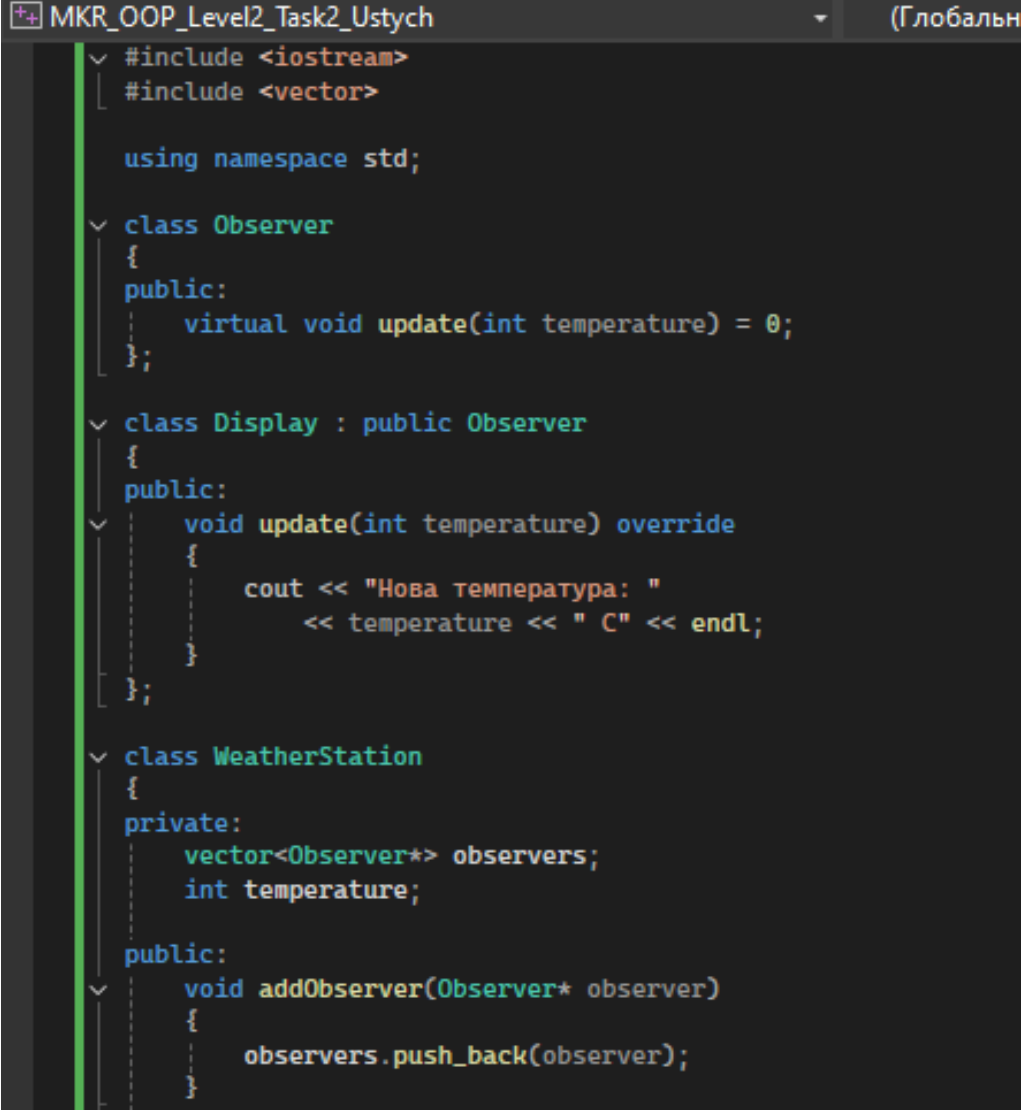
1. Новий стан об'єкта.

2. Повідомлення про зміну даних.
3. Значення змінної.
4. Текстові повідомлення.
5. Інформацію про події системи.

Отримувачами цієї інформації є всі зареєстровані об'єкти-спостерігачі.

Наприклад, у системі прогнозу погоди після зміни температури всі дисплеї автоматично отримують нове значення температури через метод `Update()`.

### Приклад програмної реалізації:



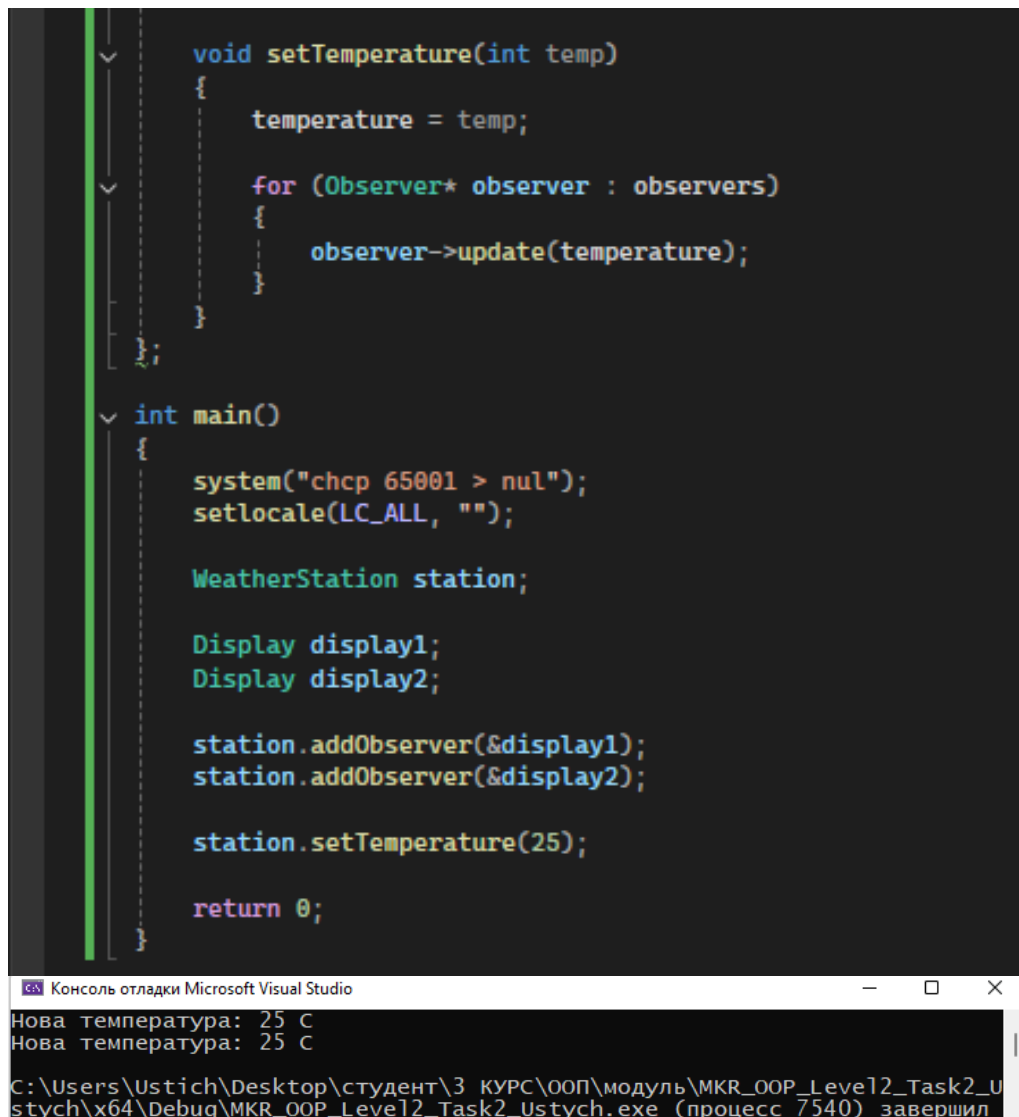
```
++ MKR_OOP_Level2_Task2_Ustych (Глобальн
#include <iostream>
#include <vector>

using namespace std;

class Observer
{
public:
    virtual void update(int temperature) = 0;
};

class Display : public Observer
{
public:
    void update(int temperature) override
    {
        cout << "Нова температура: "
              << temperature << " C" << endl;
    }
};

class WeatherStation
{
private:
    vector<Observer*> observers;
    int temperature;
public:
    void addObserver(Observer* observer)
    {
        observers.push_back(observer);
    }
};
```



```
void setTemperature(int temp)
{
    temperature = temp;

    for (Observer* observer : observers)
    {
        observer->update(temperature);
    }
}

int main()
{
    system("chcp 65001 > nul");
    setlocale(LC_ALL, "");

    WeatherStation station;

    Display display1;
    Display display2;

    station.addObserver(&display1);
    station.addObserver(&display2);

    station.setTemperature(25);

    return 0;
}
```

Консоль отладки Microsoft Visual Studio

Нова температура: 25 C  
Нова температура: 25 C

C:\Users\Ustich\Desktop\студент\3 КУРС\ООП\модуль\MKR\_OOP\_Level2\_Task2\_Ustych\x64\Debug\MKR\_OOP\_Level2\_Task2\_Ustych.exe (процесс 7540) завершил

Рис. 10 – Приклад використання патерну Observer у C++.

**3. Наскільки стабільним є об'єктно-орієнтований код при додаванні нових операцій і при додаванні нових типів? Наведіть конкретний приклад програмної реалізації.**

Об'єктно-орієнтований код є достатньо стабільним при додаванні нових типів об'єктів завдяки використанню успадкування, поліморфізму та абстракції.

Нові типи можна додавати без зміни вже існуючого коду, що відповідає принципу відкритості/закритості (Open/Closed Principle).

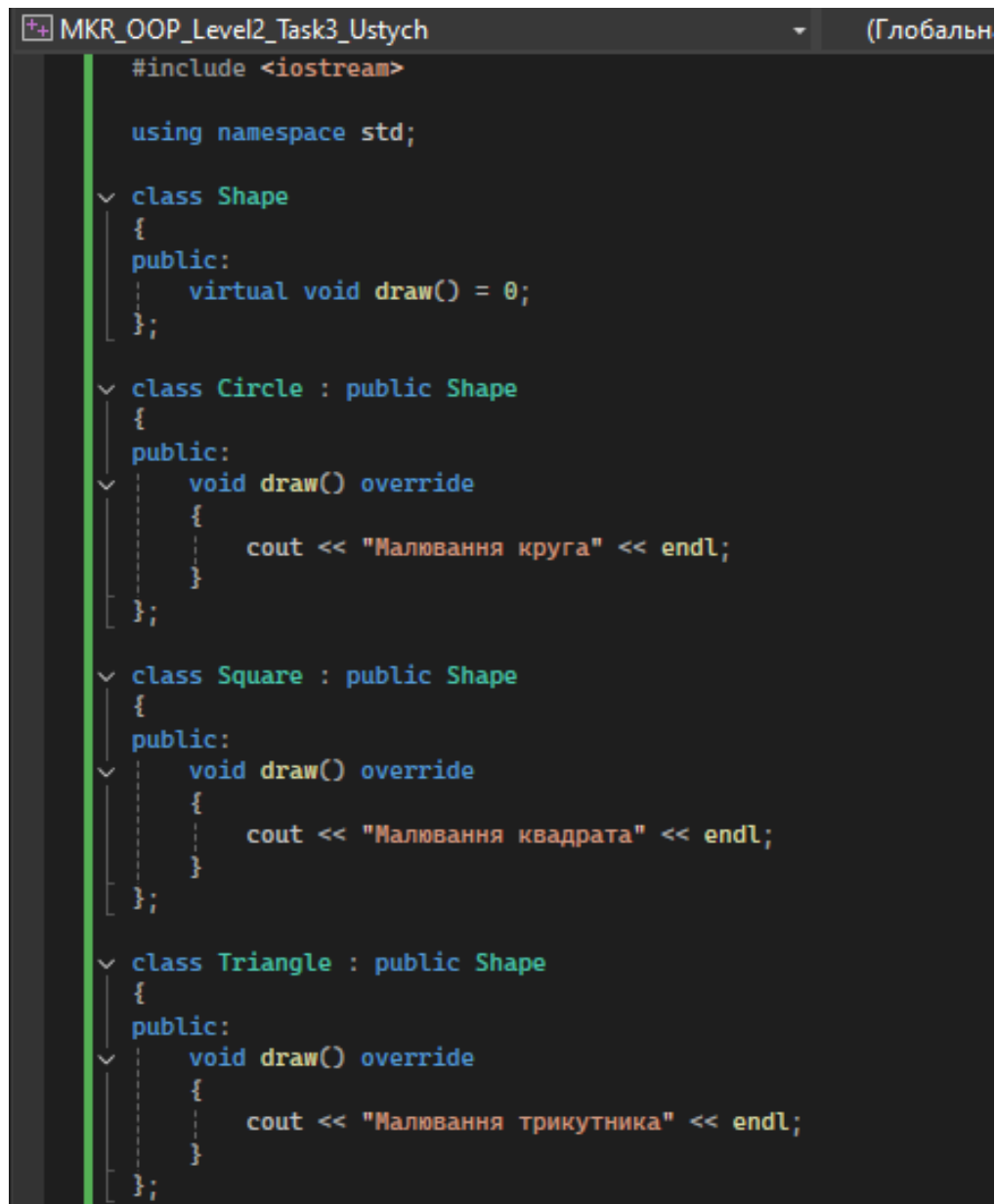
Проте при додаванні нових операцій інколи необхідно змінювати вже існуючі класи, особливо якщо операції не були передбачені заздалегідь.

Таким чином:

1. Додавання нових типів об'єктів є простішим та стабільнішим.
2. Додавання нових операцій може вимагати модифікації існуючих класів.

Наприклад, якщо у системі є базовий клас **Shape**, то можна легко додати новий клас **Triangle**, не змінюючи вже існуючі класи **Circle** або **Square**.

### Приклад програмної реалізації:



```

MKR_OOP_Level2_Task3_Ustych (Глобальн
#include <iostream>

using namespace std;

class Shape
{
public:
    virtual void draw() = 0;
};

class Circle : public Shape
{
public:
    void draw() override
    {
        cout << "Малювання круга" << endl;
    }
};

class Square : public Shape
{
public:
    void draw() override
    {
        cout << "Малювання квадрата" << endl;
    }
};

class Triangle : public Shape
{
public:
    void draw() override
    {
        cout << "Малювання трикутника" << endl;
    }
};

```

```
int main()
{
    system("chcp 65001 > nul");
    setlocale(LC_ALL, "");

    Shape* shapes[3];

    Circle c;
    Square s;
    Triangle t;

    shapes[0] = &c;
    shapes[1] = &s;
    shapes[2] = &t;

    for (int i = 0; i < 3; i++)
    {
        shapes[i]->draw();
    }

    return 0;
}
```

Консоль отладки Microsoft Visual Studio

Малювання круга  
Малювання квадрата  
Малювання трикутника

C:\Users\Ustich\Desktop\студент\3 КУРС\ООП\модуль\MKR\_OOP\_Level2\_Task3\_Ustych\x64\Debug\MKR\_OOP\_Level2\_Task3\_Ustych.exe (процесс 2120) завершил работу с кодом 0 (0x0).

Рис. 11 – Приклад стабільності об'єктно-орієнтованого коду при додаванні нових типів.

У наведеному прикладі до системи було додано новий клас `Triangle`, який успадковується від базового класу `Shape`. При цьому код класів `Circle` і `Square` не змінювався. Завдяки використанню поліморфізму всі об'єкти можуть оброблятися через один базовий тип `Shape`. Це забезпечує стабільність і гнучкість об'єктно-орієнтованого програмування.

**4. У чому полягає рекурсивність декорування згідно патерну Декоратор?  
Наведіть конкретний приклад програмної реалізації.**

Патерн `Decorator` використовується для динамічного додавання нових можливостей об'єктам без зміни їхнього коду.

Рекурсивність декорування полягає у тому, що один декоратор може містити всередині себе інший декоратор. Таким чином об'єкт може бути «обгорнутий» декількома декораторами одночасно.

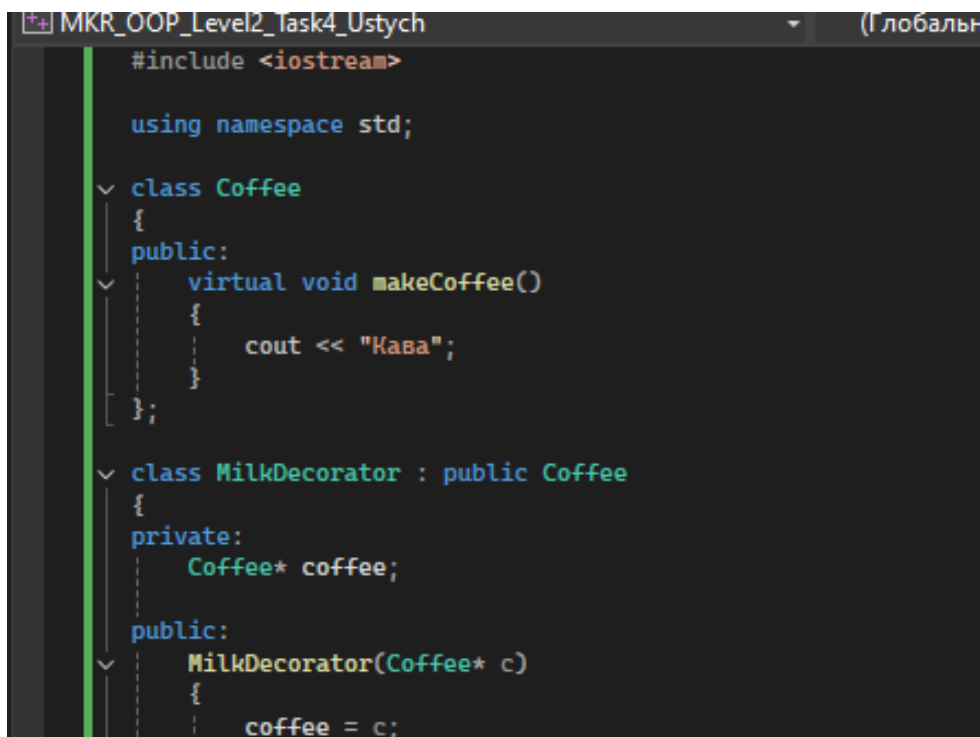
Кожен новий декоратор додає об'єкту нову поведінку або нові можливості.

Переваги патерну Decorator:

1. Динамічне розширення функціональності об'єктів.
2. Відсутність необхідності змінювати існуючий код.
3. Гнучке комбінування можливостей об'єктів.
4. Зменшення кількості похідних класів.

Наприклад, у системі приготування кави можна поступово додавати молоко, шоколад або цукор. Кожен новий декоратор розширює можливості базового об'єкта.

**Приклад програмної реалізації:**



```

MKR_OOP_Level2_Task4_Ustych (Глобальн
#include <iostream>

using namespace std;

class Coffee
{
public:
    virtual void makeCoffee()
    {
        cout << "Кава";
    }
};

class MilkDecorator : public Coffee
{
private:
    Coffee* coffee;
public:
    MilkDecorator(Coffee* c)
    {
        coffee = c;
    }
};

```



```
};

void makeCoffee() override
{
    coffee->makeCoffee();
    cout << " + молоко";
}

};

class SugarDecorator : public Coffee
{
private:
    Coffee* coffee;
public:
    SugarDecorator(Coffee* c)
    {
        coffee = c;
    }

    void makeCoffee() override
    {
        coffee->makeCoffee();
        cout << " + цукор";
    }
};

int main()
{
    system("chcp 65001 > nul");
    setlocale(LC_ALL, "");

    Coffee basicCoffee;

    MilkDecorator milkCoffee(&basicCoffee);

    SugarDecorator sweetMilkCoffee(&milkCoffee);

    sweetMilkCoffee.makeCoffee();

    return 0;
}
```

Консоль отладки Microsoft Visual Studio

Кава + молоко + цукор  
C:\Users\Ustich\Desktop\студент\3 КУРС\ООП\модуль\МКР\_ООП\_Level12\_Task4\_Ustych\h64\Debug\МКР\_ООП\_Level12\_Task4\_Ustych.exe (процесс 5740) завершил

Рис. 12 – Приклад використання патерну Decorator у C++.

У наведеному прикладі базовий об'єкт Coffee спочатку декорується класом MilkDecorator, а потім — класом SugarDecorator. Таким чином один декоратор містить інший декоратор. Це і є рекурсивність декорування. У результаті до базової кави послідовно додаються нові можливості без зміни коду основного класу.

**5. У чому полягає суть закону Деметри для ООП? Поясніть правило "не більше однієї крапки". Наведіть конкретний приклад програмної реалізації.**

Закон Деметри (Law of Demeter) — це принцип об'єктно-орієнтованого програмування, який рекомендує мінімізувати залежності між об'єктами.

Основна суть закону Деметри полягає у тому, що об'єкт повинен взаємодіяти лише зі своїми безпосередніми компонентами та не повинен знати внутрішню структуру інших об'єктів.

Цей принцип часто називають правилом «не більше однієї крапки».

Це означає, що у викликах методів небажано використовувати довгі ланцюжки звернень типу:

```
a.getB().getC().doWork();
```

Такий код створює сильну залежність між класами та ускладнює підтримку програми.

Правильніше приховувати внутрішню структуру об'єктів і надавати необхідні методи безпосередньо через основний клас.

Переваги закону Деметри:

1. Зменшення залежностей між класами.
2. Спрощення підтримки та модифікації коду.
3. Підвищення читабельності програми.
4. Покращення інкапсуляції.

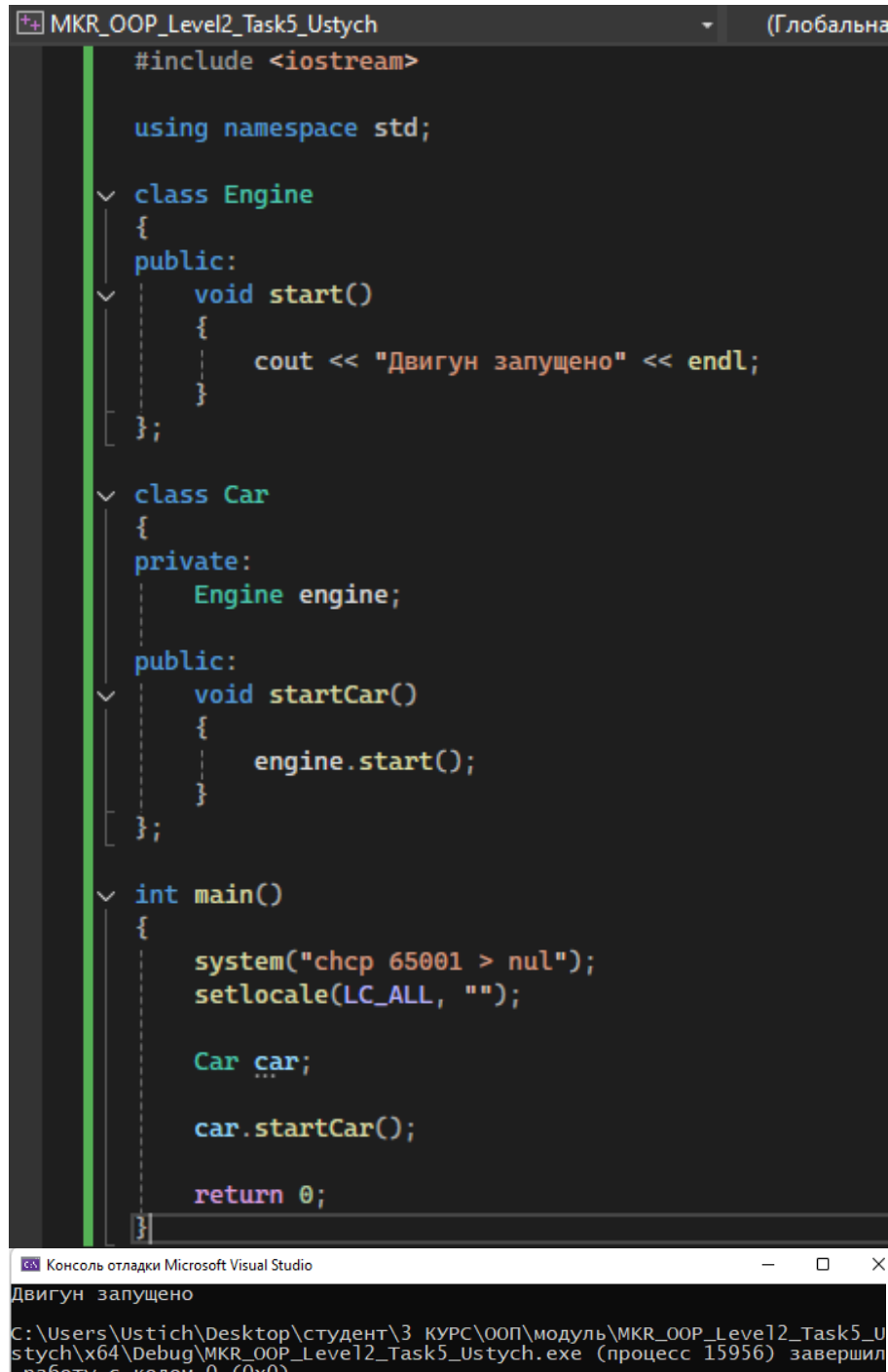
Приклад програмної реалізації:

**Неправильний варіант:**

```
car.getEngine().start();
```

**Правильний варіант:**

```
car.startCar();
```



*Рис. 13 – Приклад реалізації закону Деметри у C++.*

У наведеному прикладі клас `Car` самостійно керує роботою об'єкта `Engine`. Користувач не звертається безпосередньо до внутрішніх

компонентів автомобіля. Завдяки цьому зменшується зв'язність між класами та покращується інкапсуляція програми.

***6. Здійсніть порівняльну характеристику відомих Вам поведінкових, породжувальний і структурних патернів проектування на підставі конкретних прикладів. Результати оформіть у табличному вигляді.***

Патерни проектування — це типові рішення поширених проблем при розробленні програмного забезпечення. Вони поділяються на три основні групи:

1. Породжувальні патерни — відповідають за створення об'єктів.
2. Структурні патерни — визначають способи побудови структури класів та об'єктів.
3. Поведінкові патерни — визначають взаємодію між об'єктами.

Порівняльна характеристика патернів проектування наведена у таблиці.

| Тип патерну    | Назва патерну  | Призначення                                     | Приклад використання                       |
|----------------|----------------|-------------------------------------------------|--------------------------------------------|
| Породжувальний | Factory Method | Створення об'єктів без прямого використання new | Створення об'єктів Dog і Cat через фабрику |
| Породжувальний | Singleton      | Створення лише одного екземпляра класу          | Клас налаштувань програми                  |

|              |           |                                             |                                                |
|--------------|-----------|---------------------------------------------|------------------------------------------------|
| Структурний  | Decorator | Динамічне розширення можливостей об'єкта    | Кава + молоко + цукор                          |
| Структурний  | Adapter   | Узгодження несумісних інтерфейсів           | Підключення старої бібліотеки до нової системи |
| Структурний  | Facade    | Спрощення складної системи                  | Єдиний інтерфейс для роботи з підсистемами     |
| Поведінковий | Observer  | Автоматичне повідомлення об'єктів про зміни | Система прогнозу погоди                        |
| Поведінковий | Visitor   | Винесення операцій за межі класів           | Робота з різними фігурами                      |
| Поведінковий | Strategy  | Заміна алгоритмів під час роботи програми   | Вибір способу оплати                           |

### Приклад програмної реалізації патерну Factory Method:

```

MKR_OOP_Level2_Task6_Ustych (Глобальн
#include <iostream>

using namespace std;

class Animal
{
public:
    virtual void sound() = 0;
};

class Dog : public Animal
{
public:
    void sound() override

```

```
        cout << "Собака гавкає" << endl;
    };
};

class AnimalFactory
{
public:
    static Animal* createDog()
    {
        return new Dog();
    }
};

int main()
{
    system("chcp 65001 > nul");
    setlocale(LC_ALL, "");

    Animal* animal;

    animal = AnimalFactory::createDog();

    animal->sound();

    delete animal;

    return 0;
}
```

Консоль отладки Microsoft Visual Studio

Собака гавкає

C:\Users\Ustich\Desktop\студент\3 КУРС\ООП\модуль\MKR\_OOP\_Level2\_Task6\_Ustych\x64\Debug\MKR\_OOP\_Level2\_Task6\_Ustych.exe (процесс 13692) завершил

Рис. 14 – Приклад використання патерну *Factory Method* у C++.

У наведеній таблиці показано основні особливості поведінкових, структурних та породжувальних патернів проектування. Кожен тип патернів вирішує окремий клас задач і використовується залежно від потреб програмного забезпечення.

**7. Поясніть на конкретному прикладі програмної реалізації, коли необхідно здійснювати заміну оператора *switch* поліморфізмом?**

Заміна оператора *switch* поліморфізмом необхідна у випадках, коли програма містить велику кількість умов для роботи з різними типами об'єктів.

При використанні `switch` при додаванні нового типу доводиться змінювати вже існуючий код. Це ускладнює підтримку програми та порушує принцип відкритості/закритості (Open/Closed Principle).

Поліморфізм дозволяє кожному класу самостійно реалізовувати потрібну поведінку через перевизначення віртуальних методів.

Недоліки використання `switch`:

1. Велика кількість умов.
2. Ускладнення підтримки коду.
3. Необхідність змінювати існуючий код при додаванні нових типів.
4. Порушення принципів ООП.

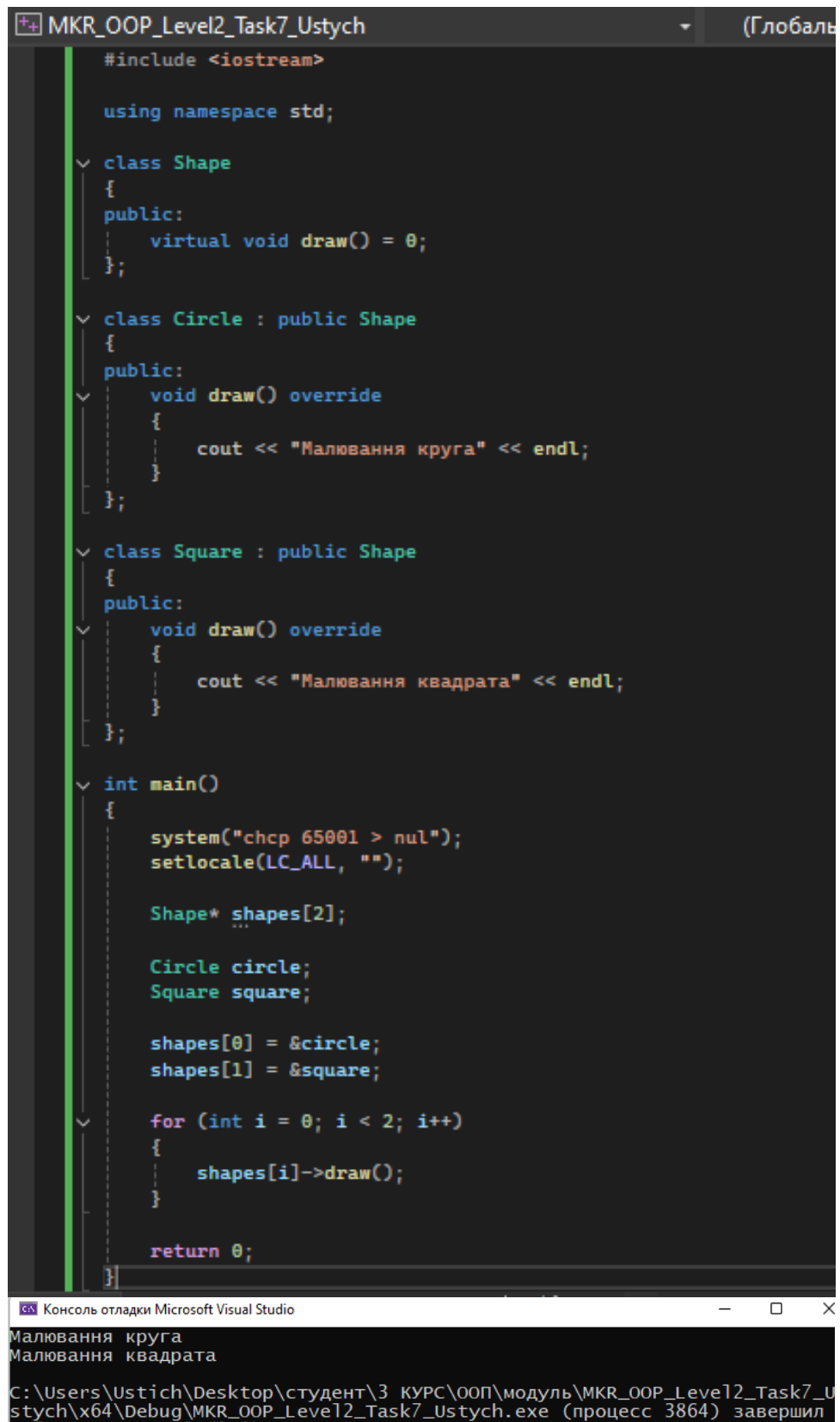
При використанні поліморфізму нові типи додаються без зміни існуючого коду.

Приклад використання оператора `switch`:

```
switch(type)
{
case 1:
cout << "Малювання круга" << endl;
break;

case 2:
cout << "Малювання квадрата" << endl;
break;
}
```

## Приклад програмної реалізації з використанням поліморфізму:



```

MKR_OOP_Level2_Task7_Ustych (Глобаль
#include <iostream>

using namespace std;

class Shape
{
public:
    virtual void draw() = 0;
};

class Circle : public Shape
{
public:
    void draw() override
    {
        cout << "Малювання круга" << endl;
    }
};

class Square : public Shape
{
public:
    void draw() override
    {
        cout << "Малювання квадрата" << endl;
    }
};

int main()
{
    system("chcp 65001 > nul");
    setlocale(LC_ALL, "");

    Shape* shapes[2];

    Circle circle;
    Square square;

    shapes[0] = &circle;
    shapes[1] = &square;

    for (int i = 0; i < 2; i++)
    {
        shapes[i]->draw();
    }

    return 0;
}

```

Консоль отладки Microsoft Visual Studio

Малювання круга  
Малювання квадрата

C:\Users\Ustych\Desktop\студент\3 КУРС\ООП\модуль\MKR\_OOP\_Level2\_Task7\_Ustych\x64\Debug\MKR\_OOP\_Level2\_Task7\_Ustych.exe (процесс 3864) завершил

Рис. 15 – Приклад заміни оператора switch поліморфізмом у C++.



У наведеному прикладі замість оператора `switch` використовується поліморфізм. Кожен клас самостійно реалізує метод `draw()`. При додаванні нового типу фігури не потрібно змінювати існуючий код, достатньо створити новий похідний клас. Це робить програму більш гнучкою, зрозумілою та зручною для підтримки.

**8. Здійсніть порівняльну характеристику функціонально-процедурного та об'єктно-орієнтованого підходів. Заповніть нижче наведену таблицю.**

Функціонально-процедурний та об'єктно-орієнтований підходи є основними моделями програмування.

Функціонально-процедурний підхід базується на використанні функцій та процедур для оброблення даних. Дані та функції існують окремо

Об'єктно-орієнтований підхід базується на використанні об'єктів і класів, які поєднують дані та методи їх оброблення.

Порівняльна характеристика підходів наведена у таблиці.

| Назва підходу             | Функції                     | Типи даних                | Команди                     | Тип моделі програмування | Рекомендації                      |
|---------------------------|-----------------------------|---------------------------|-----------------------------|--------------------------|-----------------------------------|
| Функціонально-процедурний | Основний акцент робиться на | Дані та функції розділені | Послідовне виконання команд | Процедур на модель       | Підходить для невеликих програм і |

|                           |                                                           |                                              |                               |                                     |                                                                 |
|---------------------------|-----------------------------------------------------------|----------------------------------------------|-------------------------------|-------------------------------------|-----------------------------------------------------------------|
|                           | функціях і<br>процедурах                                  |                                              |                               |                                     | простих<br>алгоритмів                                           |
| Об'єктно-<br>орієнтований | Основний<br>акцент<br>робиться на<br>класах і<br>об'єктах | Дані та<br>методи<br>об'єднані<br>в об'єктах | Взаємодія<br>між<br>об'єктами | Об'єктно-<br>орієнтова<br>на модель | Підходить<br>для великих<br>та складних<br>програмних<br>систем |

**Приклад програмної реалізації функціонально-процедурного та об'єктно-орієнтованого підходу:**

```

MKR_OOP_Level2_Task8_Ustych (Глобальная область)
#include <iostream>

using namespace std;

void printMessage()
{
    cout << "Функціонально-процедурний підхід" << endl;
}

int main()
{
    system("chcp 65001 > nul");
    setlocale(LC_ALL, "");

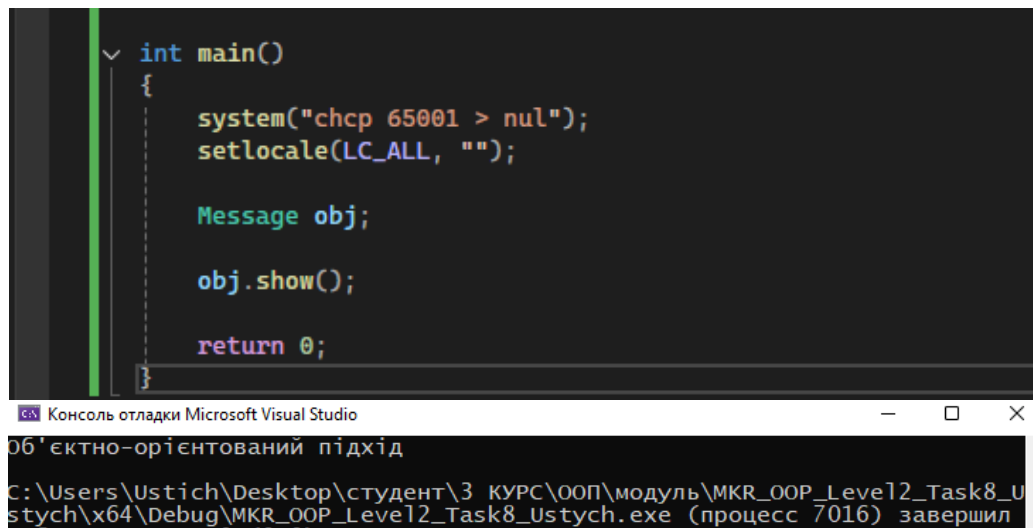
    printMessage();

    return 0;
}
Консоль отладки Microsoft Visual Studio
Функціонально-процедурний підхід
C:\Users\Ustich\Desktop\студент\3 КУРС\ООП\модуль\MKR_OOP_Level2_Task8_Ustych\x64\Debug\MKR_OOP_Level2_Task8_Ustych.exe (процесс 14724) завершил работу с кодом 0 (0x0).

MKR_OOP_Level2_Task8_Ustych (Глобальная область)
#include <iostream>

using namespace std;

class Message
{
public:
    void show()
    {
        cout << "Об'єктно-орієнтований підхід" << endl;
    }
};
  
```



```
int main()
{
    system("chcp 65001 > nul");
    setlocale(LC_ALL, "");

    Message obj;

    obj.show();

    return 0;
}
```

Консоль отладки Microsoft Visual Studio

Об'єктно-орієнтований підхід

C:\Users\Ustich\Desktop\студент\3 КУРС\ООП\модуль\МКР\_ООП\_Level12\_Task8\_Ustych\x64\Debug\МКР\_ООП\_Level12\_Task8\_Ustych.exe (процесс 7016) завершил

Рис. 16 – Порівняння функціонально-процедурного та об'єктно-орієнтованого підходів у C++.

У наведеній таблиці показано основні відмінності між функціонально-процедурним та об'єктно-орієнтованим підходами. Об'єктно-орієнтований підхід забезпечує кращу масштабованість, повторне використання коду та зручність підтримки програмного забезпечення.

## МКР Рівень 3

- 1. Створити абстрактний клас для роботи з геометричними ФІГУРАМИ на екрані. У захищеній частині класу знаходяться такі дані: координати центра фігури; кут повороту (у градусах); масштабний фактор. У відкритій частині розміщено функції-методи: зобразити фігуру на екрані; зробити фігуру невидимою; повернути фігуру на заданий кут (задається у градусах); перемістити фігуру на заданий вектор. Застосовуючи успадкування та наведений вище абстрактний клас, створити похідні класи для роботи з фігурою: трикутник, чотирикутник, багатокутник. Для перевірки пізнього зв'язування визначити масив вказівників на абстрактний клас, яким присвоюється адреси об'єктів похідних класів. Використати вказівники для виклику віртуальної функції.**

Абстрактні класи в об'єктно-орієнтованому програмуванні використовуються для створення спільного базового інтерфейсу для групи пов'язаних об'єктів.

У даній програмі створено абстрактний клас **Figure**, який містить спільні характеристики геометричних фігур: координати центра, кут повороту та масштабний фактор. Також у класі реалізовано методи для переміщення, повороту та приховування фігури.

На основі базового класу створено похідні класи **Triangle**, **Quadrangle** і **Polygon**, які перевизначають віртуальний метод **draw()**.

Для демонстрації пізнього зв'язування використовується масив вказівників на базовий клас **Figure**. Це дозволяє працювати з різними типами фігур через єдиний інтерфейс.

### Приклад програмної реалізації:

```
MKR_OOP_Level3_Task1_Ustych
Figure

#include <iostream>
using namespace std;

class Figure
{
protected:
    int x, y;
    double angle;
    double scale;

public:
    Figure(int xCoord, int yCoord, double a, double s)
    {
        x = xCoord;
        y = yCoord;
        angle = a;
        scale = s;
    }

    virtual void draw() = 0;

    virtual void hide()
    {
        cout << "Фігуру зроблено невидимою" << endl;
    }

    virtual void rotate(double degrees)
    {
        angle += degrees;
        cout << "Фігуру повернуто на " << degrees << " градусів" << endl;
    }

    virtual void move(int dx, int dy)
    {
        x += dx;
        y += dy;
        cout << "Фігуру переміщено у точку (" << x << "; " << y << ")" << endl;
    }
}
```

```

        virtual ~Figure() {}
    };

    class Triangle : public Figure
    {
    public:
        Triangle(int x, int y, double a, double s)
            : Figure(x, y, a, s) {}

        void draw() override
        {
            cout << "Зображено трикутник" << endl;
        }
    };

    class Quadrangle : public Figure
    {
    public:
        Quadrangle(int x, int y, double a, double s)
            : Figure(x, y, a, s) {}

        void draw() override
        {
            cout << "Зображено чотирикутник" << endl;
        }
    };

    class Polygon : public Figure
    {
    public:
        Polygon(int x, int y, double a, double s)
            : Figure(x, y, a, s) {}

        void draw() override
        {
            cout << "Зображено багатокутник" << endl;
        }
    };

    int main()
    {
        system("chcp 65001 > nul");
        setlocale(LC_ALL, "");

        Triangle triangle(0, 0, 0, 1);
        Quadrangle quadrangle(10, 10, 0, 1);
        Polygon polygon(20, 20, 0, 1);

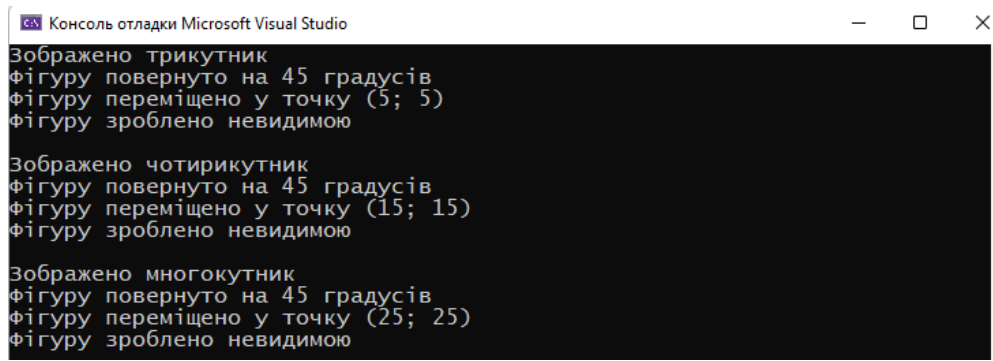
        Figure* figures[3];

        figures[0] = &triangle;
        figures[1] = &quadrangle;
        figures[2] = &polygon;

        for (int i = 0; i < 3; i++)
        {
            figures[i]->draw();
            figures[i]->rotate(45);
            figures[i]->move(5, 5);
            figures[i]->hide();
            cout << endl;
        }

        return 0;
    }

```



```
Консоль отладки Microsoft Visual Studio
Зображено трикутник
фігуру повернуто на 45 градусів
фігуру переміщено у точку (5; 5)
фігуру зроблено невидимою

Зображено чотирикутник
фігуру повернуто на 45 градусів
фігуру переміщено у точку (15; 15)
фігуру зроблено невидимою

Зображено багатокутник
фігуру повернуто на 45 градусів
фігуру переміщено у точку (25; 25)
фігуру зроблено невидимою
```

Рис. 17 – Реалізація абстрактного класу для роботи з геометричними фігурами у C++.

У наведеній програмі створено абстрактний клас `Figure`, який містить спільні характеристики геометричних фігур: координати центра, кут повороту та масштабний фактор. Метод `draw()` оголошено як чисто віртуальний, тому клас `Figure` є абстрактним.

Похідні класи `Triangle`, `Quadrangle` і `Polygon` перевизначають метод `draw()`. Для перевірки пізнього зв'язування створено масив вказівників `Figure* figures[3]`, у який записано адреси об'єктів похідних класів. Під час виклику `figures[i]->draw()` виконується метод саме того класу, на об'єкт якого вказує вказівник. Це демонструє роботу віртуальних функцій і пізнього зв'язування.

**2. Цифровий лічильник - це змінна з обмеженим діапазоном, яка скидається у початкове значення, коли її цілочисельне значення досягає визначного максимуму. Опишіть клас такого ЛІЧИЛЬНИКА. Забезпечте можливість встановлення максимального і мінімального значень, збільшення значень лічильника на 1, повернення поточного значення.**

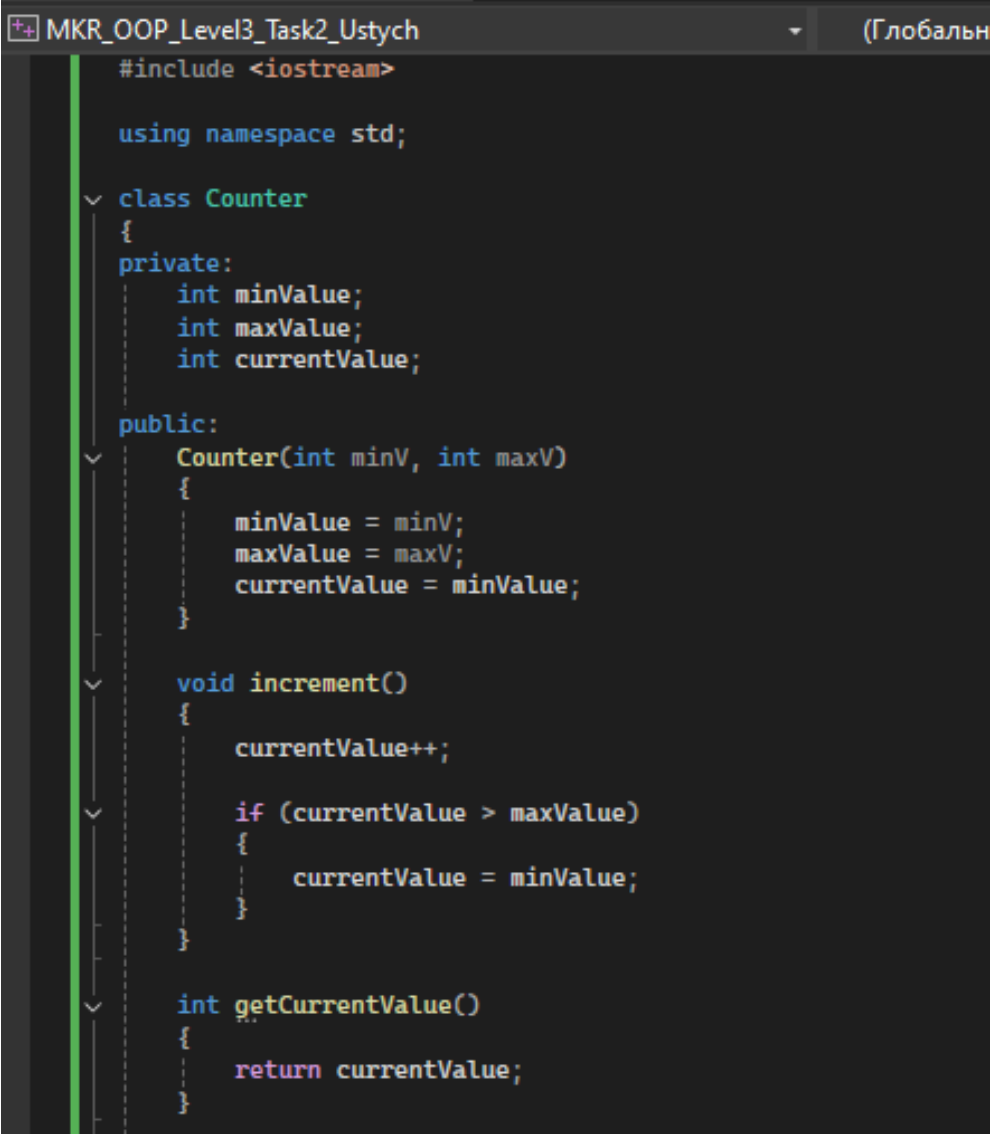
Цифровий лічильник — це змінна з обмеженим діапазоном значень, яка автоматично повертається до початкового значення після досягнення максимального значення.

У даній програмі створено клас Counter, який дозволяє:

- встановлювати мінімальне значення;
- встановлювати максимальне значення;
- збільшувати значення лічильника на 1;
- отримувати поточне значення лічильника.

Якщо значення лічильника досягає максимального значення, воно автоматично скидається до мінімального.

**Приклад програмної реалізації:**



```
MKR_OOP_Level3_Task2_Ustych (Глобальн
#include <iostream>

using namespace std;

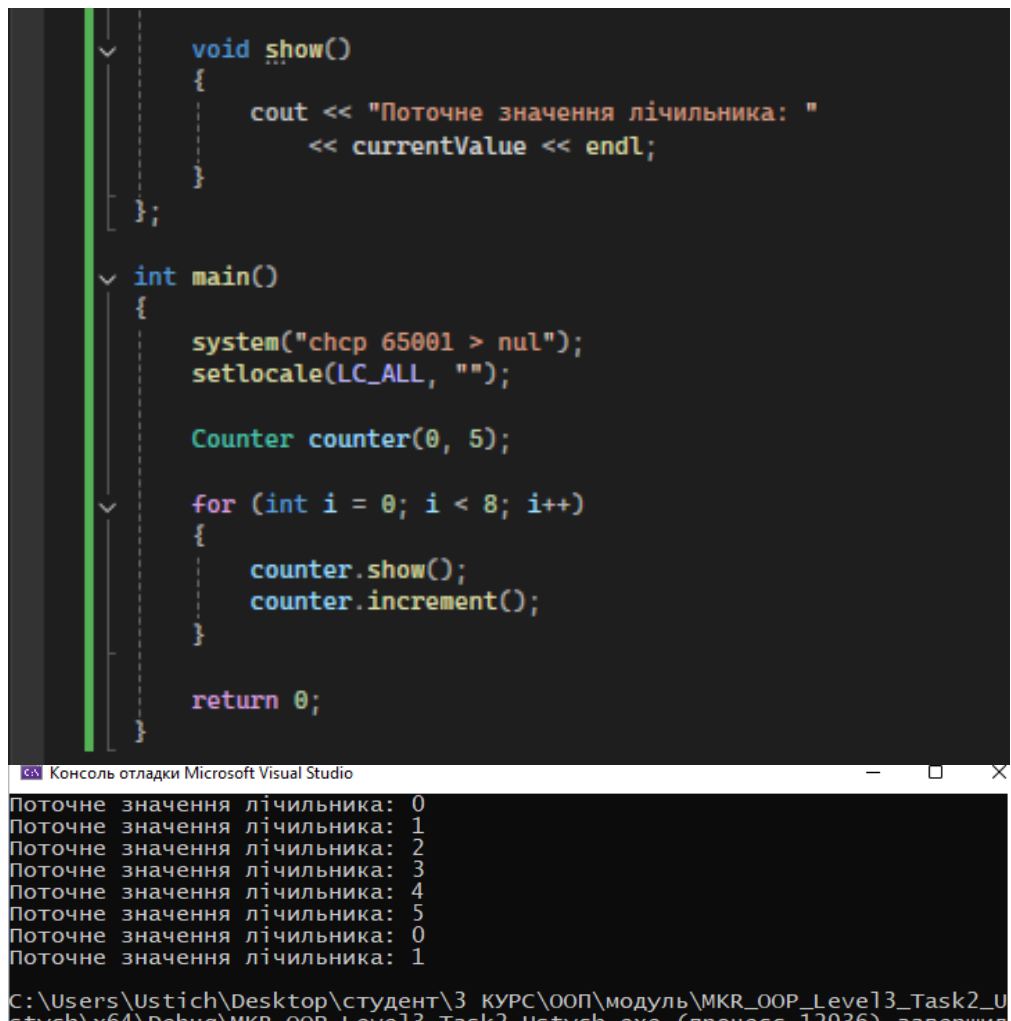
class Counter
{
private:
    int minValue;
    int maxValue;
    int currentValue;

public:
    Counter(int minV, int maxV)
    {
        minValue = minV;
        maxValue = maxV;
        currentValue = minValue;
    }

    void increment()
    {
        currentValue++;

        if (currentValue > maxValue)
        {
            currentValue = minValue;
        }
    }

    int getCurrentValue()
    {
        return currentValue;
    }
}
```



```
void show()
{
    cout << "Поточне значення лічильника: "
        << currentValue << endl;
}

};

int main()
{
    system("chcp 65001 > nul");
    setlocale(LC_ALL, "");

    Counter counter(0, 5);

    for (int i = 0; i < 8; i++)
    {
        counter.show();
        counter.increment();
    }

    return 0;
}
```

Консоль отладки Microsoft Visual Studio

Поточне значення лічильника: 0  
Поточне значення лічильника: 1  
Поточне значення лічильника: 2  
Поточне значення лічильника: 3  
Поточне значення лічильника: 4  
Поточне значення лічильника: 5  
Поточне значення лічильника: 0  
Поточне значення лічильника: 1

C:\Users\Ustich\Desktop\студент\3 КУРС\ООП\модуль\MKR\_OOP\_Level3\_Task2\_Ustich\x64\Debug\MKR\_OOP\_Level3\_Task2\_Ustich.exe (process 12036) завершил

Рис. 18 – Реалізація класу цифрового лічильника у C++.

У наведеній програмі створено клас Counter, який реалізує роботу цифрового лічильника. Лічильник має мінімальне та максимальне значення. Метод increment() збільшує поточне значення на 1. Якщо значення перевищує максимальне, лічильник автоматично повертається до початкового значення. Це демонструє роботу циклічного цифрового лічильника.

**3. Написати шаблонний клас, який створює копію двовимірного динамічного масиву довільного типу. Передбачити операції копіювання за рядками, за стовпцями, копіювання вибраного рядка або стовпчика, копіювання головної діагоналі. Розробити функції потокового введення виведення масивів.**

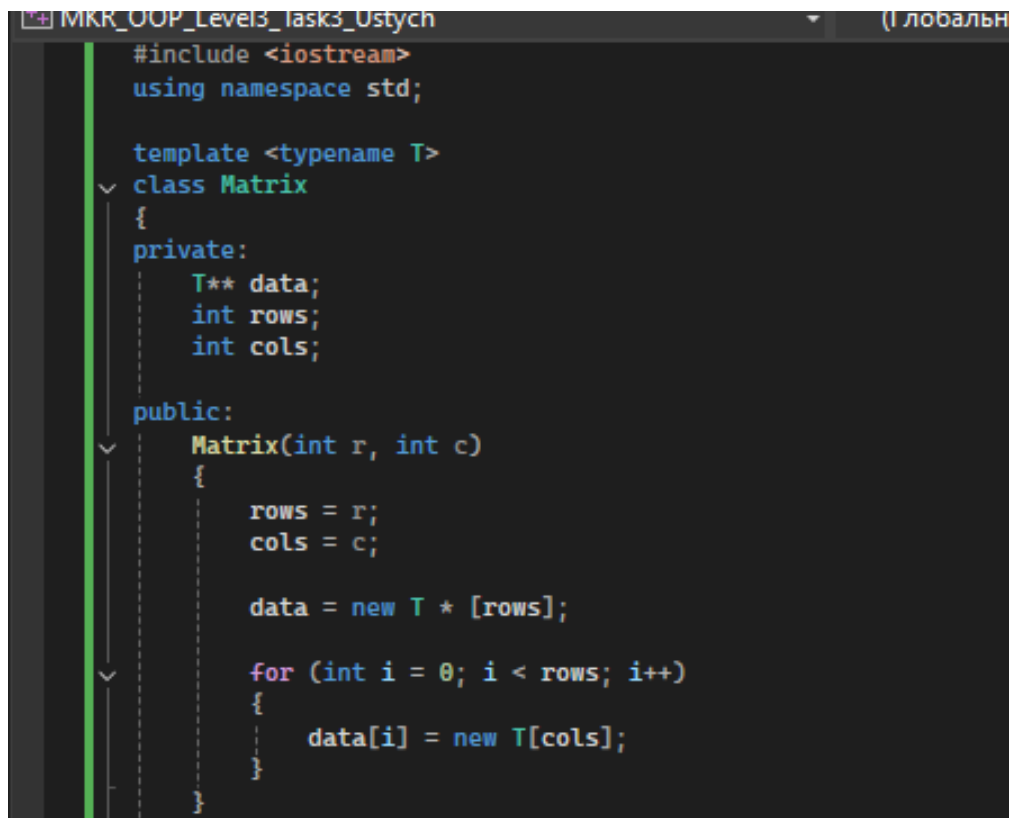


Шаблонні класи в C++ використовуються для роботи з даними різних типів. У даній програмі створено шаблонний клас `Matrix`, який працює з двовимірним динамічним масивом довільного типу.

Клас дозволяє виконувати такі операції:

1. створення двовимірного динамічного масиву;
2. введення та виведення елементів масиву;
3. копіювання масиву за рядками;
4. копіювання масиву за стовпцями;
5. копіювання вибраного рядка;
6. копіювання вибраного стовпчика;
7. копіювання головної діагоналі.

**Приклад програмної реалізації:**



```
MRK_OOP_Level3_task3_Ustych (Глобальн)

#include <iostream>
using namespace std;

template <typename T>
class Matrix
{
private:
    T** data;
    int rows;
    int cols;

public:
    Matrix(int r, int c)
    {
        rows = r;
        cols = c;

        data = new T * [rows];

        for (int i = 0; i < rows; i++)
        {
            data[i] = new T[cols];
        }
    }
}
```

```

~Matrix()
{
    for (int i = 0; i < rows; i++)
    {
        delete[] data[i];
    }

    delete[] data;
}

void input()
{
    cout << "Введіть елементи матриці:" << endl;

    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            cin >> data[i][j];
        }
    }
}

void output()
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            cout << data[i][j] << " ";
        }

        cout << endl;
    }
}

void copyByRows()
{
    cout << "Копіювання за рядками:" << endl;

    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            cout << data[i][j] << " ";
        }

        cout << endl;
    }
}

void copyByColumns()
{
    cout << "Копіювання за стовпцями:" << endl;

    for (int j = 0; j < cols; j++)
    {

```

```

        for (int i = 0; i < rows; i++)
        {
            cout << data[i][j] << " ";
        }

        cout << endl;
    }
}

void copyRow(int row)
{
    cout << "Копіювання вибраного рядка:" << endl;

    for (int j = 0; j < cols; j++)
    {
        cout << data[row][j] << " ";
    }

    cout << endl;
}

void copyColumn(int col)
{
    cout << "Копіювання вибраного стовпчика:" << endl;

    for (int i = 0; i < rows; i++)
    {
        cout << data[i][col] << " ";
    }

    cout << endl;
}

void copyMainDiagonal()
{
    cout << "Копіювання головної діагоналі:" << endl;

    int n = rows < cols ? rows : cols;

    for (int i = 0; i < n; i++)
    {
        cout << data[i][i] << " ";
    }

    cout << endl;
}

};

int main()
{
    system("chcp 65001 > nul");
    setlocale(LC_ALL, "");

    Matrix<int> matrix(3, 3);

    matrix.input();

    cout << endl << "Початкова матриця:" << endl;
    matrix.output();
}

```

```
cout << endl;
matrix.copyByRows();

cout << endl;
matrix.copyByColumns();

cout << endl;
matrix.copyRow(1);

cout << endl;
matrix.copyColumn(2);

cout << endl;
matrix.copyMainDiagonal();

return 0;
}
```

Консоль отладки Microsoft Visual Studio

Введіть елементи матриці:  
1 2 3  
4 5 6  
7 8 9

Початкова матриця:  
1 2 3  
4 5 6  
7 8 9

Копіювання за рядками:  
1 2 3  
4 5 6  
7 8 9

Копіювання за стовпцями:  
1 4 7  
2 5 8  
3 6 9

Копіювання вибраного рядка:  
4 5 6

Копіювання вибраного стовпчика:  
3 6 9

Копіювання головної діагоналі:  
1 5 9

C:\Users\Ustich\Desktop\студент\3 КУРС\ООП\модуль\МКР\_ООП\_Level3\_Task3\_Ustych\Debug\MKR\_ООП\_Level3\_Task3\_Ustych.exe (процесс 2560) завершил

Рис. 19 – Реалізація шаблонного класу для копіювання двовимірного динамічного масиву у C++.

У наведеній програмі створено шаблонний клас `Matrix`, який дозволяє працювати з двовимірним динамічним масивом довільного типу. Пам'ять для масиву виділяється динамічно за допомогою оператора `new`, а звільняється у деструкторі за допомогою `delete[]`.

Методи класу дозволяють виводити матрицю, копіювати її за рядками та стовпцями, копіювати вибраний рядок, вибраний стовпчик і головну

діагональ. Завдяки використанню шаблону `template <typename T>` цей клас можна застосовувати не лише для цілих чисел, а й для інших типів даних.

*4. Написати програму з використанням функції, яка знаходить суму своїх аргументів і повертає результат через перший аргумент. Реалізувати варіанти функції, яка дозволяє звернення з різною кількістю аргументів:*

*а) перевантаження функцій;*

*б) функція з параметрами за замовчуванням;*

*в) функція зі змінною кількістю параметрів  $f(int \&, \dots)$ .*

*Результати функції повернути через вказівник та через посилання.*

У даній програмі реалізовано функції для знаходження суми аргументів різними способами:

1. перевантаження функцій;
2. використання параметрів за замовчуванням;
3. використання функції зі змінною кількістю параметрів.

Результат роботи функцій повертається через посилання та через вказівник.

**Приклад програмної реалізації:**

+

+

MRK\_OOP\_Level3\_Task4\_Ustych

▼

(Глобальн

▼

#include <iostream>

|

#include <cstdlib>

using namespace std;

▼

void sum(int& result, int a, int b)

|

{

|

result = a + b;

|

}

▼

void sum(int& result, int a, int b, int c)

|

{

|

result = a + b + c;

|

}

▼

void sumDefault(int\* result, int a, int b = 0, int c = 0)

|

{

|

\*result = a + b + c;

|

}

▼

void sumVariable(int& result, int count, ...)

|

{

|

va\_list args;

|

va\_start(args, count);

|

result = 0;

▼

for (int i = 0; i < count; i++)

|

{

|

result += va\_arg(args, int);

|

}

|

va\_end(args);

|

}

▼

int main()

|

{

|

system("chcp 65001 > nul");

|

setlocale(LC\_ALL, "");

|

int result1;

|

int result2;

|

int result3;

|

sum(result1, 5, 10);

|

cout << "Сума двох чисел: "

|

<< result1 << endl;

|

sum(result1, 5, 10, 15);

|

cout << "Сума трьох чисел: "

|

<< result1 << endl;

|

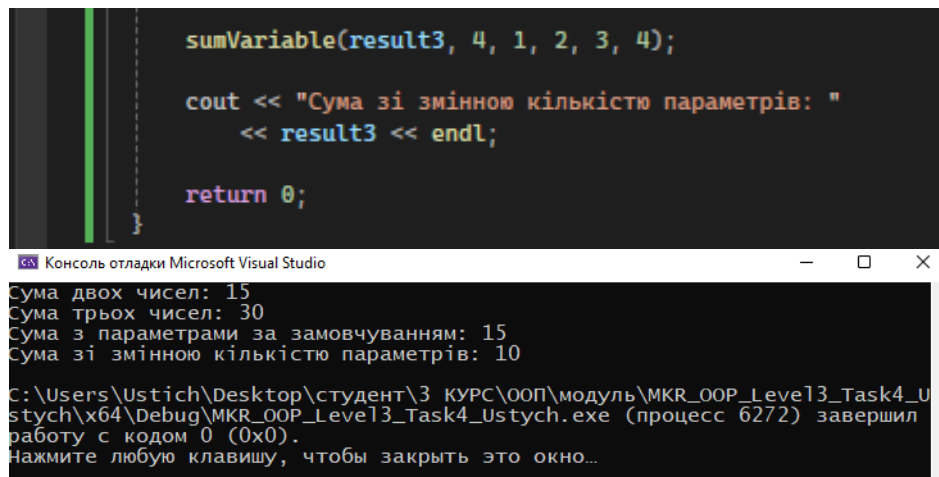
sumDefault(&result2, 7, 8);

|

cout << "Сума з параметрами за замовчуванням: "

|

<< result2 << endl;



```
sumVariable(result3, 4, 1, 2, 3, 4);

cout << "Сума зі змінною кількістю параметрів: "
    << result3 << endl;

return 0;
}
```

Консоль отладки Microsoft Visual Studio

Сума двох чисел: 15  
Сума трьох чисел: 30  
Сума з параметрами за замовчуванням: 15  
Сума зі змінною кількістю параметрів: 10

C:\Users\Ustich\Desktop\студент\3 КУРС\ООП\модуль\MKR\_OOP\_Level13\_Task4\_Ustych\x64\Debug\MKR\_OOP\_Level13\_Task4\_Ustych.exe (процесс 6272) завершил работу с кодом 0 (0x0).  
Нажмите любую клавишу, чтобы закрыть это окно...

Рис. 20 – Реалізація функцій для знаходження суми аргументів у C++.

У наведеній програмі використано різні способи реалізації функцій для обчислення суми аргументів. Перевантаження функцій дозволяє створювати декілька функцій з однаковим ім'ям, але різними параметрами. Параметри за замовчуванням спрощують виклик функцій, а функція зі змінною кількістю параметрів дозволяє працювати з довільною кількістю аргументів. Результати роботи повертаються через посилання та через вказівник.